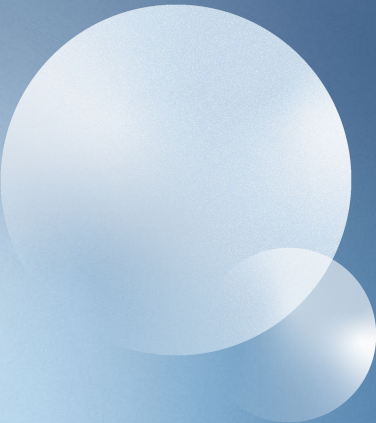




CSC343Y5

LECTURE 4

DML Continued



ADMIN

- Checkpoint 1 grades out
 - Resubmission due Monday, June 1st

MAIN CHANGES FOR CHECKPOINT 2

- Justifications:
 - This should be richer - justify every decision clearly
 - Why did you choose the cardinality?
 - Why did you choose the participation constraints?
 - What about the specifications made you model the ERD this way?

MAIN CHANGES FOR CHECKPOINT 2

- ERD diagrams should be cleaner
 - No overlapping lines
 - No lines not touching entities
 - Many pictures were blurry
- Was a bit more lenient with this in the first round

CHECKPOINT 2 DDL

- Some relationships are attributes (1:1, 1:m) to make the DDL simpler
 - Just complete the todos :)
- This will make querying in the next checkpoint much easier!

GAME TIME!! :)

Besalf won last time - who will this time?

FORMAL SEMANTICS

Almost the same for single-relation queries:

- Start with the **product** of all the relations in the **FROM** clause
- Apply the selection condition from the **WHERE** clause
- Project onto the list of attributes and expressions in the **SELECT** clause

FROM
choose the table



WHERE
filter the rows



SELECT
choose the columns



RESULT
the final output

FORMAL SEMANTICS

```
SELECT name, major
FROM Students
WHERE year = 2;
```

① FROM

start with this table

Students

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

② WHERE

keep rows that satisfy
the condition

Students (year = 2)

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

③ SELECT

keep only the
needed columns

Result

name	major
Ben	Math
Chloe	CS

FROM
choose the table

WHERE
filter the rows

SELECT
choose the columns

RESULT
the final output

OPERATIONAL SEMANTICS

Almost the same for single-relation queries:

- Start with the **product** of all the relations in the **FROM** clause
- Apply the selection condition from the **WHERE** clause
- Project onto the list of attributes and expressions in the **SELECT** clause

FROM
choose the table



WHERE
filter the rows



SELECT
choose the columns



RESULT
the final output

WHICH STUDENTS ARE ENROLLED IN CSC108?

sid	course
1	CSC108
1	MAT136
3	CSC108

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

WHICH STUDENTS ARE ENROLLED IN CSC108?

sid	course
1	CSC108
1	MAT136
3	CSC108

How would you describe the procedure you just did in your head?

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

WHICH STUDENTS ARE ENROLLED IN CSC108?

sid	course
1	CSC108
1	MAT136
3	CSC108

Does this pair belong together? -
WHERE

What do we print from this? -
SELECT

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

OPERATIONAL SEMANTICS

Thinking about how SQL processes queries - we pretend the database walks through every combination of rows, checks each one, and keeps the good ones.

The real database doesn't actually work that way, it's much smarter and faster. But the recipe is guaranteed to give the same answer, so it's a reliable story you can think with.

Tldr: operational semantics tells you what a query means by giving you a procedure that produces its answer.

Tuple-variable t1

1

sid	course
1	CSC108
1	MAT136
3	CSC108

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

Tuple-variable t1

Check course name

1

sid	course
1	CSC108
1	MAT136
3	CSC108

2

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

Tuple-variable t1

Check course name

1	sid	course	2
	1	CSC108	
	1	MAT136	
	3	CSC108	

Tuple-variable t2

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

Tuple-variable t1

Check course name

Tuple-variable t2

1	sid	course	2
	1	CSC108	
	1	MAT136	
	3	CSC108	4

	sid	name	gpa	3
	1	Anya	3.8	
	2	Boris	3.2	
	3	Carmen	3.9	

Check these are equal

Tuple-variable t1

Check course name

Tuple-variable t2

sid	course
1	CSC108
1	MAT136
3	CSC10

sid	name	gpa
1	Anya	3.8
2	Boris	
3	Carmen	3.9

Check these are equal

To output

Tuple-variable t1

Check course name

Tuple-variable t2

1

sid	course
1	CSC108
1	MAT136
3	CSC108

2

4

Check these are equal

sid

name

gpa

1

Anya

3.8

2

Boris

3

Carmen

3.9

3

5

To output

EXPLICIT TUPLE-VARIABLES

Sometimes a query needs to use two copies of the same relation. Copies are distinguished by following the relation name by **the name of a tuple-variable**, in the FROM clause.

It's always an option to rename relations this way, even when not essential.

```
SELECT  s1.name, s2.name
FROM    Students s1, Students s2
WHERE   s1.course = s2.course
AND     s1.name < s2.name;
```


SUB-QUERIES

A parenthesized **SELECT-FROM-WHERE** statement (subquery) can be used as a value in a number of places, including **FROM** and **WHERE** clauses.

Example: in place of a relation in the FROM clause, we can use a subquery and then query its result.

To do this we must use a tuple-variable to name tuples of the result.

EXAMPLE: SUB-QUERY IN FROM

```
SELECT  course
FROM    Takes,
        (SELECT sid
         FROM Students
         WHERE program = 'CS') CSStudents
WHERE   Takes.sid = CSStudents.sid;
```

Students who are in
the CS program



EXAMPLE: SUB-QUERY IN FROM

① Start with Students

Students

sid	name	program
1	Ava	CS
2	Ben	Math
3	Chloe	CS
4	Diego	Bio

↑
Rows where program = 'CS'
are highlighted.

```
SELECT  course
FROM    Takes,
        (SELECT sid
         FROM Students
         WHERE program = 'CS') CSStudents
WHERE   Takes.sid = CSStudents.sid;
```


EXAMPLE: SUB-QUERY IN FROM

① Start with Students

Students

sid	name	program
1	Ava	CS
2	Ben	Math
3	Chloe	CS
4	Diego	Bio

↑
Rows where program = 'CS'
are highlighted.

② Run the subquery first

Subquery (inside)

```
SELECT sid  
FROM Students  
WHERE program = 'CS'
```

Result: temporary table

CSStudents

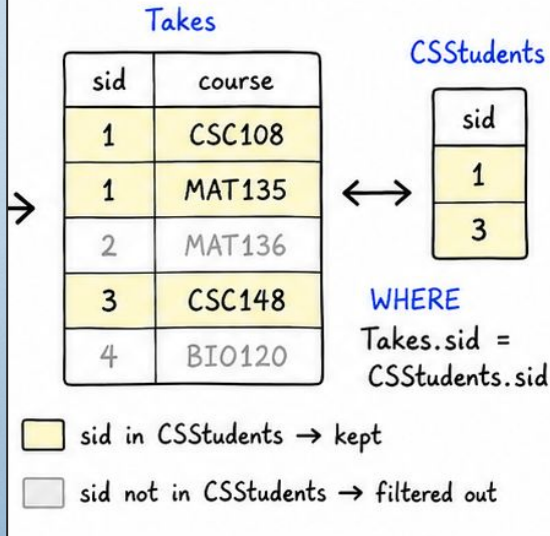
sid
1
3

(temporary)

```
SELECT course  
FROM Takes,  
    (SELECT sid  
     FROM Students  
     WHERE program = 'CS') CSStudents  
WHERE Takes.sid = CSStudents.sid;
```

EXAMPLE: SUB-QUERY IN FROM

③ Match with Takes



```
SELECT  course
FROM    Takes,
        (SELECT sid
         FROM Students
         WHERE program = 'CS') CSStudents
WHERE   Takes.sid = CSStudents.sid;
```

EXAMPLE: SUB-QUERY IN FROM

④ Outer SELECT
keeps only course

Final result

course
CSC108
MAT135
CSC148

```
SELECT course
FROM Takes,
      (SELECT sid
        FROM Students
        WHERE program = 'CS') CSStudents
WHERE Takes.sid = CSStudents.sid;
```



```

SELECT course
FROM Takes,
     (SELECT sid
      FROM Students
      WHERE program = 'CS') CSStudents
WHERE Takes.sid = CSStudents.sid;

```

① Start with Students

Students

sid	name	program
1	Ava	CS
2	Ben	Math
3	Chloe	CS
4	Diego	Bio

Rows where program = 'CS' are highlighted.

② Run the subquery first

Subquery (inside)

```

SELECT sid
FROM Students
WHERE program = 'CS'

```

Result: temporary table

CSStudents

sid
1
3

(temporary)

③ Match with Takes

Takes

sid	course
1	CSC108
1	MAT135
2	MAT136
3	CSC148
4	BIO120

CSStudents

sid
1
3

WHERE
Takes.sid =
CSStudents.sid

sid in CSStudents → kept

sid not in CSStudents → filtered out

④ Outer SELECT keeps only course

Final result

course
CSC108
MAT135
CSC148

Only the course column is returned.



Think of the subquery as making a temporary table first.

Students
all student data



Subquery result
CSStudents (sid 1, 3)



Match with Takes
Takes.sid = CSStudents.sid



Keep course
final output

SUB-QUERIES AS VALUES

If a subquery is **guaranteed** to return *one* tuple, then the sub-query can be used as a value.

Usually this tuple has one component.

Example:

```
SELECT  cname
FROM    Courses
WHERE   credits = ( SELECT MAX(credits)
                   FROM Courses );
```

- ① Run the subquery
(it returns one tuple
with one attribute)

Result of subquery

MAX(credits)
5

← one tuple (one row)
with one component
(e.g., value 5)

ACTIVITY

ORDERING THE DISPLAY OF TUPLES

List in alphabetical order the names of all instructors

```
SELECT name  
FROM Instructors  
ORDER BY name ASC;
```

We may specify desc for descending order or asc for ascending order, for each attribute. The default is ascending.

WHAT'S THE DIFFERENCE BETWEEN THESE?

```
SELECT gpa FROM Students
```

VS.

```
SELECT DISTINCT gpa FROM Students
```

BAG SEMANTICS



A *bag* (or *multi-set*) is like a set, but an element may appear more than once.

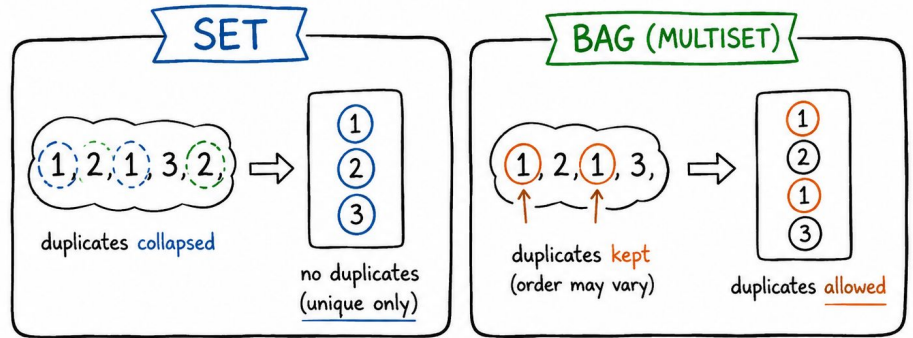


TABLE: Students

id	program
1	CS
2	CS
3	Math
4	Bio

SELECT program
FROM Students;



CS
CS
Math
Bio

BAG result
(duplicates kept)

SELECT DISTINCT program
FROM Students;

CS
Math
Bio

SET result
(unique only)

BAG (MULTI-SET SEMANTICS)

- SQL primarily uses bag semantics
- The SELECT-FROM-WHERE statement uses bag semantics

The default for union, intersection, and difference is set semantics

- i.e. duplicates are eliminated before the operation

HOW DO WE GET THE AVERAGE GPA?

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

HOW DO WE GET THE AVERAGE GPA?

Aggregation!

```
SELECT AVG(gpa)  
FROM Students
```

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

AGGREGATION OVERVIEW

<u>Function Syntax</u>	<u>Function Usage</u>
COUNT ([ALL DISTINCT] expression)	The number of (distinct) non-NULL values in a column/expression.
COUNT(*)	The number of selected rows.
AVG ([ALL DISTINCT] expression)	The average of the (distinct) values in a numeric column/expression.
SUM ([ALL DISTINCT] expression)	The total of the (distinct) values in a numeric column/expression.
MAX (expression)	The largest value in a column/expression.
MIN (expression)	The smallest value in a column/ expression.

AGGREGATION RULES

1. Aggregation functions can be used in both the SELECT and HAVING clauses (the HAVING clause will be discussed at another time).
2. Aggregation functions cannot be used in a WHERE clause.



Aggregation is a **column procedure**, not a row procedure.

EXAMPLE: COUNT

```
SELECT COUNT(*)  
FROM Takes  
WHERE course = 'CSC108';
```

① Start with table
Takes

sid	course
1	CSC108
2	MAT135
3	CSC108
4	CSC148
5	CSC108

② Rows that match WHERE
(course = 'CSC108')

sid	course	
1	CSC108	✓
2	MAT135	×
3	CSC108	✓
4	CSC148	×
5	CSC108	✓

③ Result
COUNT(*)

COUNT(*)
3

↑
Single number
(not the rows!)



KEY IDEA: COUNT(*) counts how many rows remain **AFTER** the WHERE filter. Output is a single number, not the full rows.



FROM

table



WHERE

filters rows



COUNT(*)

returns one number

3

EXAMPLE COUNT

```
SELECT COUNT(DISTINCT course)
FROM Enrolled
```

sid	course
1	CS101
1	MATH200
3	CS101
3	CS101

GROUP BY CLAUSE

```
SELECT major,  
AVG(gpa)  
FROM Students  
GROUP BY major
```

sid	name	major	gpa
1	Anya	CS	3.8
2	Boris	CS	3.2
3	Carmen	Math	3.9
4	Diego	Math	3.5
5	Elif	Math	3.7
6	Farid	Bio	3.4

GROUP BY CLAUSE

```
SELECT major, AVG(gpa)
FROM Students
GROUP BY major
```

sid	name	major	gpa
1	Anya	CS	3.8
2	Boris	CS	3.2
3	Carmen	Math	3.9
4	Diego	Math	3.5
5	Elif	Math	3.7
6	Farid	Bio	3.4

major	AVG(gpa)
CS	3.50
Math	3.70
Bio	3.40